



ÁRVORES BINÁRIAS II - AVL

ESTRUTURA DE DADOS

CST em Desenvolvimento de Software Multiplataforma



PROF. Me. TIAGO A. SILVA



LIVRO DE REFERÊNCIA DA DISCIPLINA

- **BIBLIOGRAFIA BÁSICA:**

- GRONER, Loiane. **Estrutura de dados e algoritmos com JavaScript**: escreva um código JavaScript complexo e eficaz usando a mais recente ECMAScript. **São Paulo: Novatec Editora, 2019.**

- **NESTA AULA:**

- Capítulo 10 - Árvores



PARA SOBREVIVER AO JAVASCRIPT

Non-zero value



null



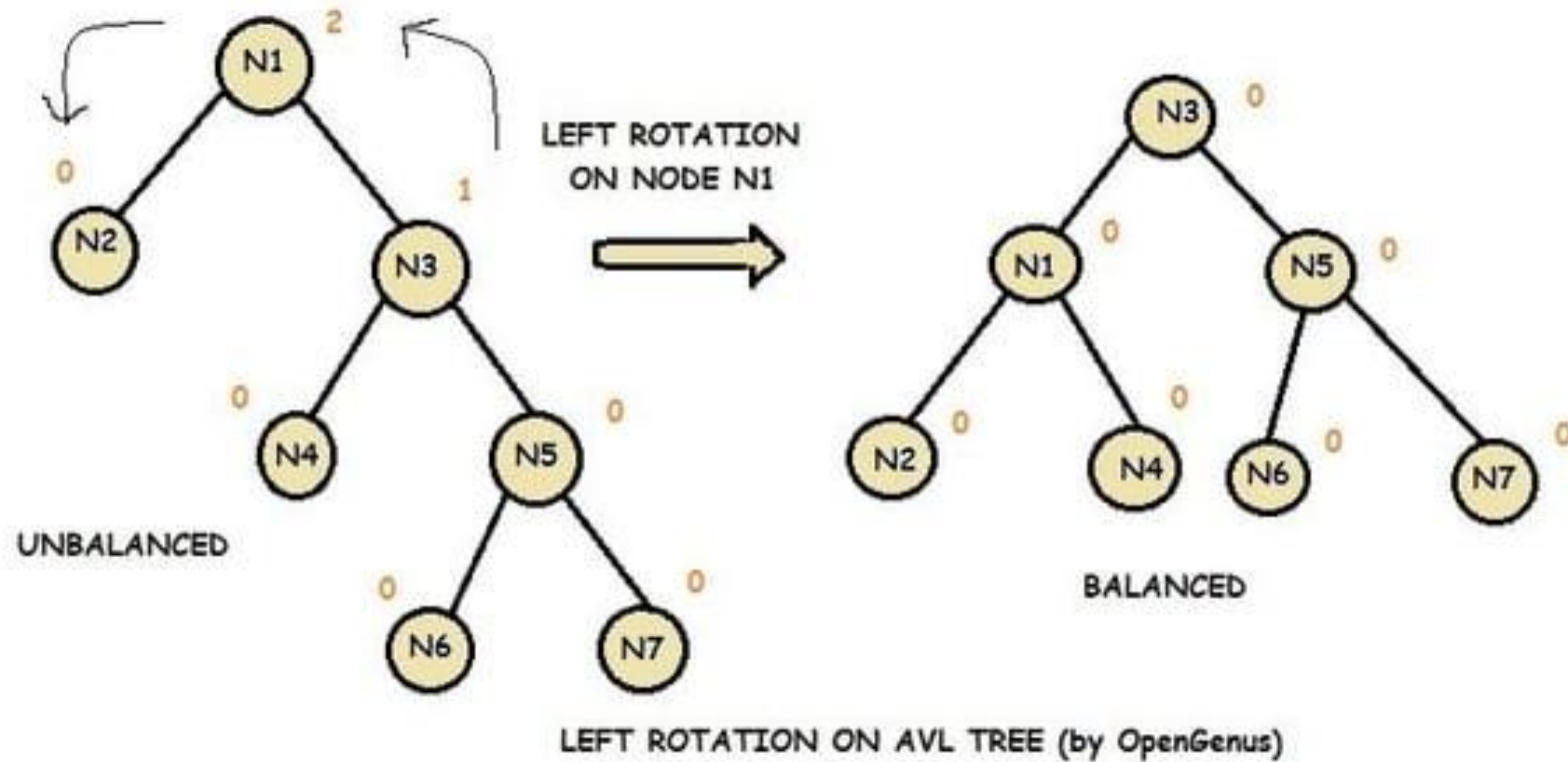
0



undefined



O QUE É UMA ÁRVORE AVL?



O QUE É UMA ÁRVORE AVL?

- A Árvore AVL (nomeada em homenagem a seus criadores Adelson-Velsky e Landis, em 1962) é um tipo de árvore binária de busca (BST) que se auto-balanceia.
- **Objetivo:**
 - Manter a árvore sempre equilibrada para garantir que as operações básicas – inserção, remoção e busca – tenham complexidade $O(\log n)$.
- **Por que precisamos de balanceamento?**
 - Em uma árvore binária de busca comum, se inserirmos os elementos em ordem crescente (ou decrescente), a árvore vira uma lista ligada degenerada, como:
- A AVL evita isso reequilibrando a árvore automaticamente.

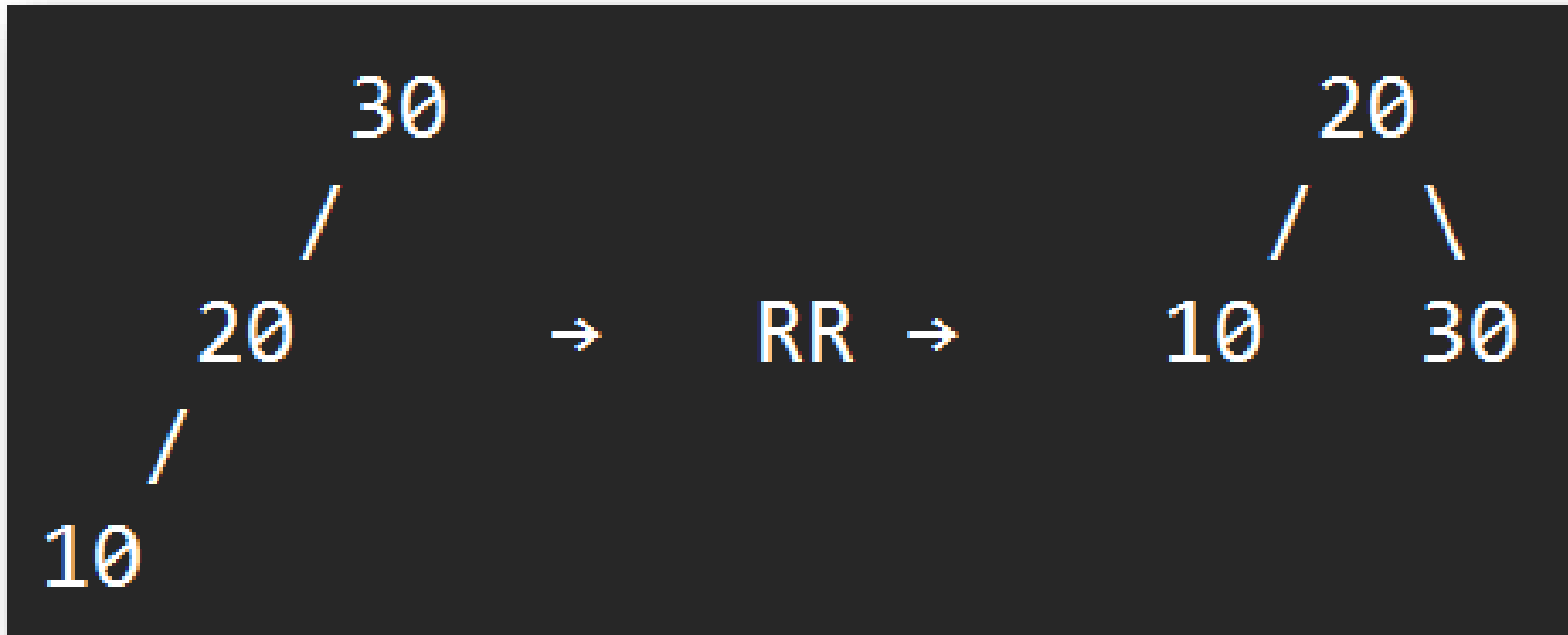
COMO A AVL SE EQUILIBRA?

- Cada nó de uma árvore AVL guarda um valor chamado fator de balanceamento (FB):
- **$FB = \text{altura}(\text{subarvore esquerda}) - \text{altura}(\text{subarvore direita})$**
- Condição de balanceamento:
 - O fator de balanceamento de cada nó deve estar sempre entre -1 e +1.
 - Se $FB = 0$ → Subárvores esquerda e direita têm a mesma altura.
 - Se $FB = +1$ → Subárvore esquerda tem uma altura a mais.
 - Se $FB = -1$ → Subárvore direita tem uma altura a mais.
 - Se $FB > 1$ ou $FB < -1$ → Desbalanceamento detectado → aplicar rotação.

TIPOS DE ROTAÇÃO

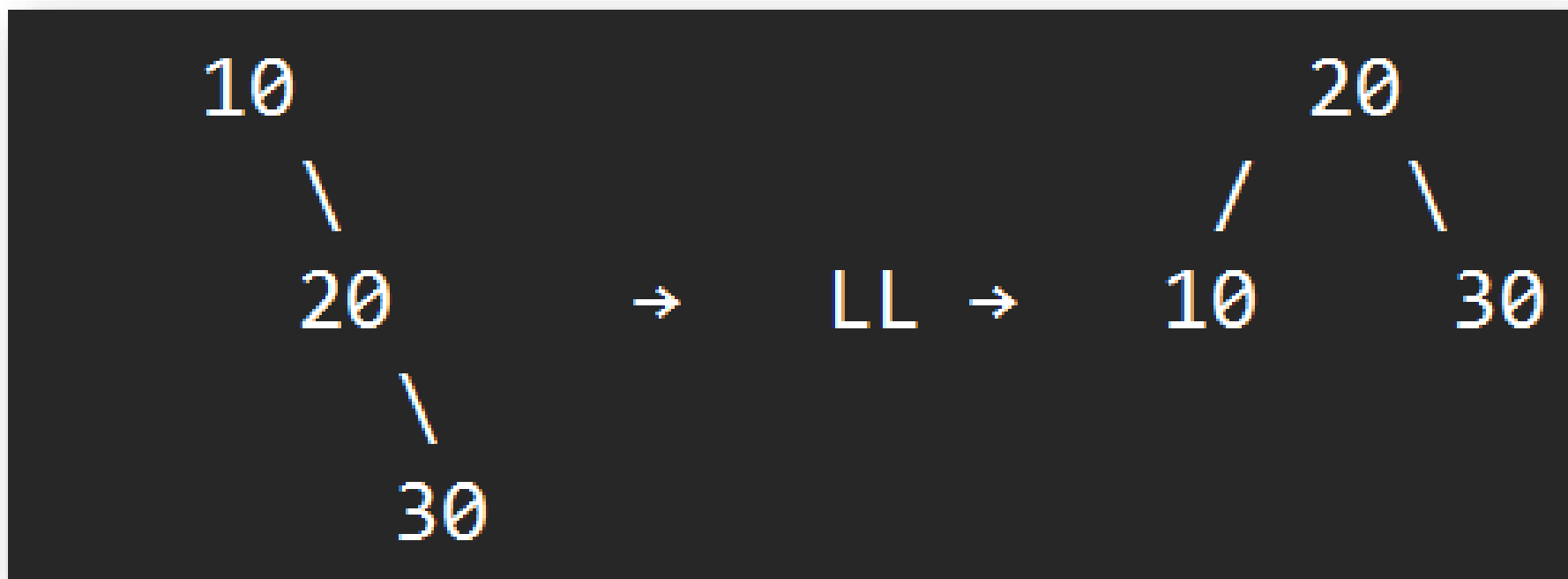
ROTAÇÃO SIMPLES À DIREITA (RR)

- Quando inserimos um nó na esquerda da subárvore esquerda:



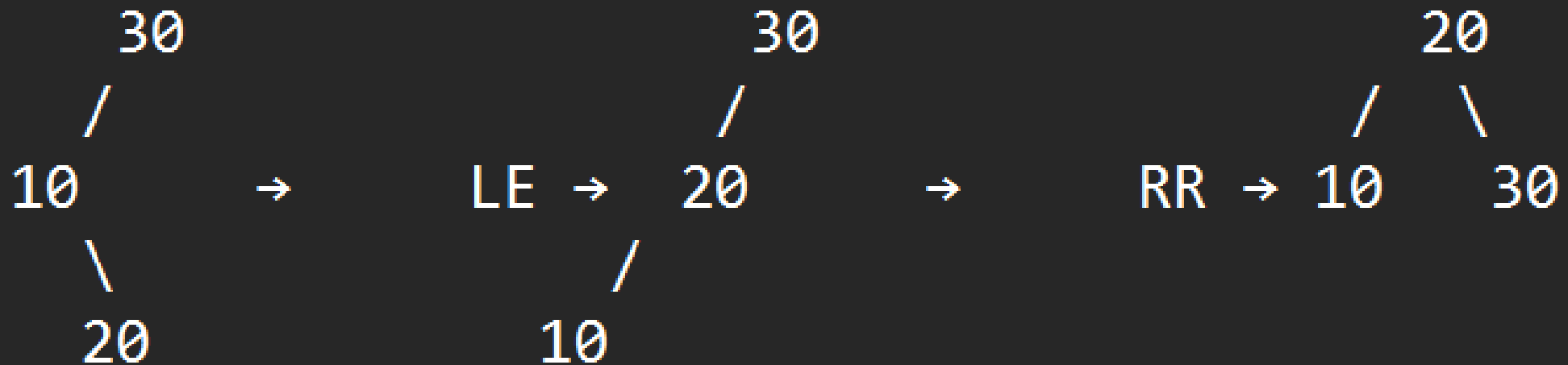
ROTAÇÃO SIMPLES À ESQUERDA (LL)

- Quando inserimos na direita da subárvore direita.



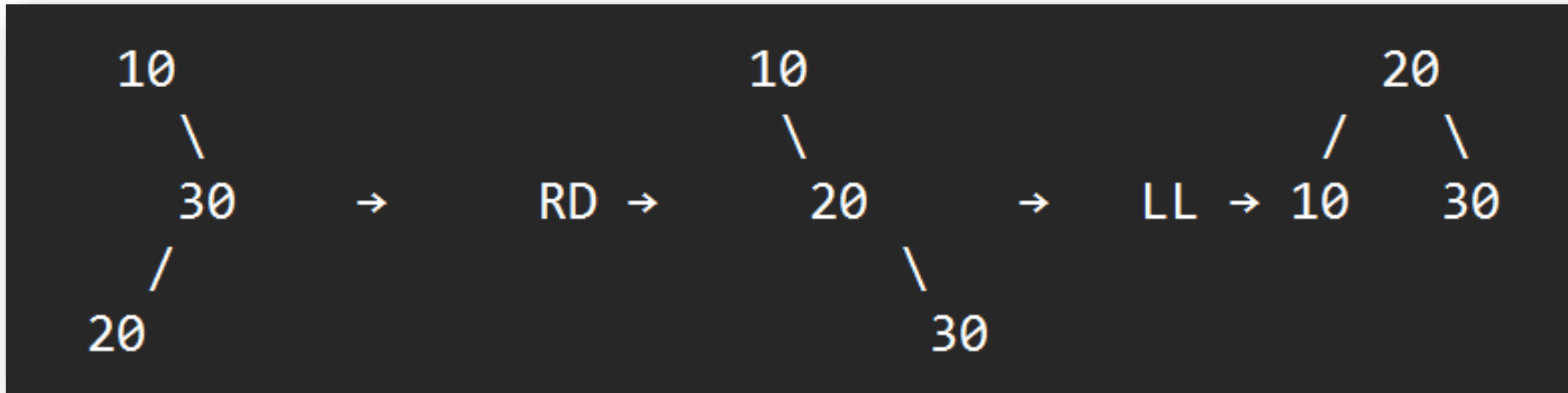
ROTAÇÃO DUPLA ESQUERDA-DIREITA

- Inserimos na direita da subárvore esquerda.



ROTAÇÃO DUPLA DIREITA-ESQUERDA

- Inserimos na esquerda da subárvore direita.



VANTAGENS E DESVANTAGENS

VANTAGENS	DESVANTAGENS
Busca rápida garantida ($\log n$)	Inserção e remoção um pouco mais complexas
Sempre balanceada	Overhead de rotações
Boa para leitura intensa (muitos acessos)	Pode ser mais lenta que outras árvores autoajustáveis como Red-Black em cenários específicos

ONDE USAMOS ÁRVORES AVL?

- ✓ Indexação de dados (banco de dados, memória).
- ✓ Árvores de busca onde as operações precisam ser rápidas e previsíveis.
- ✓ Jogos e sistemas onde é importante manter resposta rápida sob inserções frequentes.

IMPLEMENTAÇÃO DOS NÓS DA ÁRVORE

NÓS DA ÁRVORE AVL

```
1  class AVLNode {  
2      constructor(value) {  
3          this.value = value;  
4          this.left = null;  
5          this.right = null;  
6          this.height = 1; // todo nó começa com altura 1  
7      }  
8  }
```



IMPLEMENTAÇÃO DA CLASSE AVL TREE

CONSTRUTOR DA CLASSE AVLTREE

```
10 ~ class AVLTree {  
11  
12 ~     constructor() {  
13         this.root = null;  
14     }
```



OBTER A ALTURA DE UM NÓ

- **Por que é importante?**

- Altura é usada para calcular o fator de balanceamento e detectar desbalanceamento.
- Se o nó for `null`, retorna `0`.
- Se existir, retorna a altura armazenada no nó.

```
15
16 // Função utilitária para obter a altura de um nó
17 getHeight(node) {
18     return node ? node.height : 0;
19 }
```

CALCULAR O FATOR DE BALANCEAMENTO DE UM NÓ

- **Por que é importante?**
 - Esse valor indica se o nó está equilibrado (-1, 0 ou +1) ou precisa de rotação (<-1 ou >1).
- Subtrai a altura da subárvore direita da esquerda.
- Valores fora de [-1, +1] indicam desbalanceamento.

```
21 // Calcula o fator de balanceamento de um nó
22 getBalanceFactor(node) {
23     return node ? this.getHeight(node.left) - this.getHeight(node.right) : 0;
24 }
25
```

ATUALIZANDO A ALTURA DA ÁRVORE

- **Objetivo:** Atualizar a altura do nó com base nas alturas dos filhos.
- **Por que é importante?**
 - Depois de inserir ou rotacionar, a altura pode mudar e precisa ser recalculada.
- A altura de um nó é 1 (ele mesmo) + a maior altura entre seus filhos.

```
26 // Atualiza a altura de um nó
27 updateHeight(node) {
28     node.height = 1 + Math.max(this.getHeight(node.left), this.getHeight(node.right));
29 }
```

CORREÇÃO DO DESBALANCEAMENTO: ESQUERDA-ESQUERDA

- **Quando usar?**
 - Quando **getBalanceFactor(node) > 1** e o valor a ser inserido está na subárvore esquerda do filho esquerdo.
- **x** vira a nova raiz da subárvore.
- **y** passa a ser o filho direito de **x**.
- **T2**, que era o filho direito de **x**, vira o filho esquerdo de **y**.

```
31 // Rotação simples à direita
32 rotateRight(y) {
33     const x = y.left;
34     const T2 = x.right;
35
36     x.right = y;
37     y.left = T2;
38
39     this.updateHeight(y);
40     this.updateHeight(x);
41
42     return x;
43 }
```

CORREÇÃO DO DESBALANCEAMENTO: DIREIRA-DIREITA

- Quando usar?
 - Quando **getBalanceFactor(node)** < -1 e o valor está na subárvore direita do filho direito.
- **y** vira a nova raiz da subárvore.
- **x** passa a ser o filho esquerdo de **y**.
- **T2**, que era o filho esquerdo de **y**, vira o filho direito de **x**.

```
45 // Rotação simples à esquerda
46 rotateLeft(x) {
47     const y = x.right;
48     const T2 = y.left;
49
50     y.left = x;
51     x.right = T2;
52
53     this.updateHeight(x);
54     this.updateHeight(y);
55
56     return y;
57 }
```

INSERINDO NÓS NA ÁRVORE AVL

- Chama a função recursiva `_insert()` a partir da raiz.
- Substitui a raiz (se for o caso) após possíveis rotações.

```
59 // Inserção com balanceamento AVL
60 insert(value) {
61     this.root = this._insert(this.root, value);
62 }
```

INSERIR UM NÓ NA ÁRVORE AVL

```
64  _insert(node, value) {
65      if (!node) return new AVLNode(value);
66
67      if (value < node.value) {
68          node.left = this._insert(node.left, value);
69      } else if (value > node.value) {
70          node.right = this._insert(node.right, value);
71      } else {
72          return node; // valor duplicado não é inserido
73      }
74
75      this.updateHeight(node);
76      const balance = this.getBalanceFactor(node);
77
78      // Casos de desbalanceamento:
79      if (balance > 1 && value < node.left.value) {
80          return this.rotateRight(node); // Esquerda-Esquerda
81      }
```



INSERIR UM NÓ NA ÁRVORE AVL

```
82
83     if (balance < -1 && value > node.right.value) {
84         return this.rotateLeft(node); // Direita-Direita
85     }
86
87     if (balance > 1 && value > node.left.value) {
88         node.left = this.rotateLeft(node.left);
89         return this.rotateRight(node); // Esquerda-Direita
90     }
91
92     if (balance < -1 && value < node.right.value) {
93         node.right = this.rotateRight(node.right);
94         return this.rotateLeft(node); // Direita-Esquerda
95     }
96
97     return node;
98 }
```



PERCURSO IN-ORDER DA ÁRVORE (ESQUERDA → RAIZ → DIREITA)

- **Utilidade:** Mostra os valores da árvore AVL em ordem crescente.

```
100 // Exibir percurso in-order (opcional)
101 inOrder(node = this.root) {
102     if (node) {
103         this.inOrder(node.left);
104         console.log(node.value);
105         this.inOrder(node.right);
106     }
107 }
108 }
109
110 module.exports = AVLTree;
```



EXERCÍCIOS

EXERCÍCIO 1 – INSERÇÃO SIMPLES E BALANCEAMENTO LL

- Insira os valores 30, 20, 10 em uma árvore AVL vazia.
 - Mostre a árvore após cada inserção.
 - Indique o fator de balanceamento de cada nó.
 - Diga qual rotação é aplicada (se houver).
 - Desenhe a árvore final.
- **Objetivo:** identificar e corrigir desbalanceamento do tipo esquerda-esquerda (LL).



EXERCÍCIO 2 – ROTAÇÕES SIMPLES E DUPLAS

- Insira os valores 20, 10, 30, 25, 40, 22 em ordem.
 - Mostre o fator de balanceamento após cada inserção.
 - Identifique e aplique as rotações necessárias.
 - Desenhe a árvore final balanceada.
- **Objetivo:** provocar um caso esquerda-direita (LR) e aplicar uma rotação dupla.



EXERCÍCIO 3 – ROTAÇÕES MÚLTIPLAS E AVL COM DESENHO

- Dado o seguinte código:

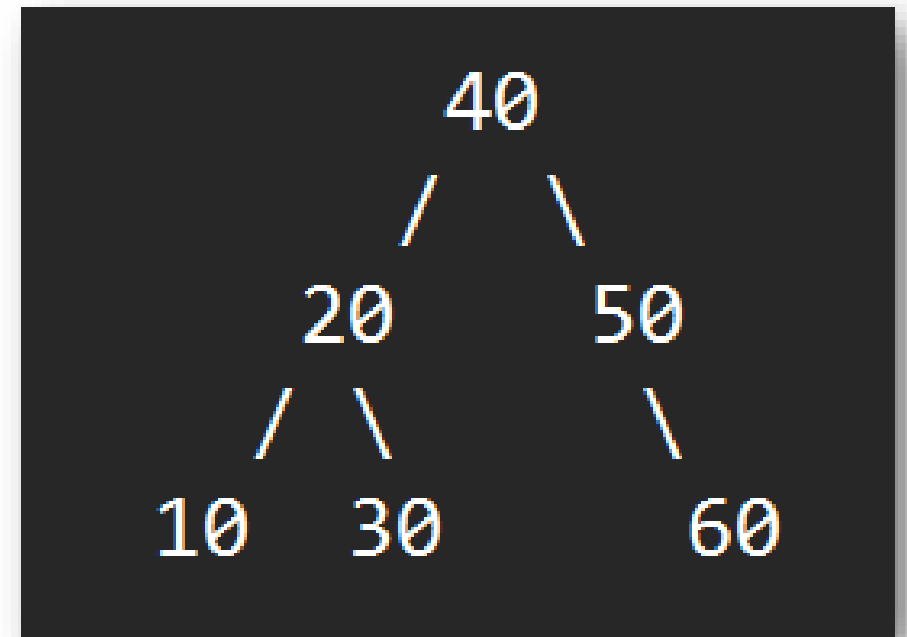
```
const avl = new AVLTree();
```

```
[50, 20, 60, 10, 30, 25, 27].forEach(v => avl.insert(v));
```

- Desenhe a árvore após cada inserção.
 - Identifique os nós desbalanceados.
 - Indique as rotações feitas (nome, tipo e onde).
 - Mostre o percurso in-order da árvore final.
- **Objetivo:** consolidar o uso de rotações simples e duplas em sequência.

EXERCÍCIO 4 – FATOR DE BALANCEAMENTO MANUAL

- Considere a seguinte árvore AVL:
 - Calcule manualmente a altura de cada nó.
 - Calcule o fator de balanceamento de cada nó.
 - Diga se a árvore está balanceada.
 - Insira o valor 70 e verifique se será necessário rebalancear.
 - **Objetivo:** reforçar cálculo de altura e fator de balanceamento, além da avaliação de necessidade de rotação.



EXERCÍCIO 5 – AVALIAÇÃO E COMPARAÇÃO COM BST

- Insira os valores **10, 20, 30, 40, 50, 60, 70**:
 - Em uma BST comum (sem balanceamento).
 - Em uma AVLTree.
- Desenhe as duas árvores.
- Compare suas alturas.
- Mostre os percursos in-order de ambas.
- Explique por que a AVL é mais eficiente para operações de busca.
- **Objetivo:** comparar BST vs AVL na prática.

OBRIGADO!

- Encontre este **material on-line** em:
 - Slides: Google Classroom
- Em caso de **dúvidas**, entre em contato:
 - **Prof. Tiago:** tiago.silva@cps.sp.gov.br

